

Atelier Lens 백엔드 2인 분업 및 구현 순서

FastAPI / PostgreSQL / 해커톤 MVP 실행 계획

권장 역할: Backend A = Core & Commerce, Backend B = Experience & AI Integration

1. 분업 원칙

백엔드 API 개수를 반으로 나누지 않고 **도메인 소유권**으로 나눈다. 목표는 두 사람이 같은 파일/테이블을 동시에 수정하는 일을 줄이고, 각자 독립적으로 개발한 뒤 명확한 Service 계약으로 연결하는 것이다.

Backend A - Core & Commerce

서비스의 계정/상품/구매/보유제품 데이터를 축을 담당한다.

인증/게스트
↓
회원/상품
↓
장바구니
↓
모의결제/주문
↓
보유제품/기본 케어

Backend B - Experience & AI Integration

사용자 체험과 파일/비동기/AI Provider 축을 담당한다.

품번 사진 OCR
↓
아바타/사진 Try-on
↓
추천/코디 저장
↓
AI 손상진단
↓
수리 예약

2. 공통으로 먼저 확정할 것 - 약 1~2시간

둘이 코딩을 시작하기 전에 아래만 같이 만든다.

- API 명세서 v3.0 확인
- `.env.example` key 이름
- 공통 response envelope
- 공통 error code
- UUID / camelCase / KST 규칙
- Git branch / PR 규칙
- SQLAlchemy Base / DB session 방식
- 도메인간 Service 호출 규칙

이후 공통 파일은 가능하면 Backend A를 owner로 두고 Backend B가 필요할 때 PR로 변경한다.

3. Backend A 담당 - Core & Commerce

3.1 파일 소유권

```
app/core/  
app/api/dependencies/auth.py  
app/modules/auth/  
app/modules/guests/  
app/modules/users/  
app/modules/products/  
app/modules/cart/  
app/modules/orders/  
app/modules/owned_products/  
app/modules/stores/  
app/providers/payments/  
app/providers/social/
```

3.2 API 담당

영역	API
게스트	POST /guest-sessions
인증	/auth/signup, /auth/login, /auth/social, /auth/refresh, /auth/logout, /auth/claim
회원	GET/PATCH /me, PATCH /me/password
상품	GET /products/{id}, GET /products/{id}/variants, GET /recent-products 의 DB 부분
장바구니	POST/GET/PATCH/DELETE /cart...
주문	POST /orders, GET /me/orders, GET /me/orders/{id}
기존/구매 제품	POST/GET /me/products, 상세, 기본 care-guide
매장 데이터	GET /stores 의 Store 조회 기반

3.3 DB Entity 담당

```
User  
RefreshToken  
GuestSession  
QrCodeMapping  
Campaign  
Store  
Product  
ProductVariant  
ProductImage  
ProductTag  
RecentProduct  
Cart  
CartItem  
Order  
OrderItem  
Payment
```

3.4 핵심 구현 포인트

Auth

- bcrypt
- JWT Access 2시간
- Refresh 14일
- Refresh Token hash 저장
- rotation/logout revoke
- 로그인 5회 실패 잠금
- Google/Kakao Authorization Code 검증
- LOCAL/GOOGLE/KAKAO 동일 이메일 별도 계정

Product

Backend B가 OCR 결과로 상품을 찾을 수 있도록 다음 Service 계약을 먼저 만든다.

```
class ProductService:
    async def find_by_product_code(self, product_code: str): ...
    async def get_product(self, product_id: UUID): ...
    async def get_available_variants(self, product_id: UUID): ...
```

B는 Product 모델/Repository를 직접 수정하지 않고 이 계약을 사용한다.

Order / Mock Payment

```
Cart 검증
→ 현재 가격/재고 조회
→ Order PENDING
→ PaymentProvider.pay()
→ Mock 성공
→ Order PAID
→ 재고 차감
→ CartItem 삭제
→ RegisteredProduct 생성
```

실제 PG가 추가되어도 PaymentProvider 만 교체할 수 있도록 한다.

4. Backend B 담당 - Experience & AI Integration

4.1 파일 소유권

```
app/modules/ocr/
app/modules/avatars/
app/modules/try_on/
app/modules/recommendations/
app/modules/coordis/
app/modules/jobs/
app/modules/files/
app/modules/diagnoses/
app/modules/repairs/
app/providers/ocr/
app/providers/try_on/
app/providers/diagnosis/
```

app/providers/recommendations/
app/storage/
app/tasks/

4.2 API 담당

영역	API
OCR	POST /product-recognitions
게스트 신체정보	PUT /guest-sessions/me/avatar-parameters
아바타	POST/GET/PUT /me/avatar
아바타 피팅	POST /avatar-try-ons
사진 피팅	POST /try-ons , 저장/목록/삭제
Job	GET /jobs/{jobId}
파일	GET /files/{fileId}
추천	GET /products/{id}/recommendations
코디	POST/GET/PATCH/DELETE /me/coordis...
진단	POST /diagnoses , 결과/케어가이드
수리	POST /repair-reservations , 내 예약, 취소

4.3 DB Entity 담당

Avatar
FileMetadata
Job
TryOn
SavedCoordi
SavedCoordiItem
Diagnosis
Damage
RepairReservation

Product , RegisteredProduct , Store 가 필요하면 A가 제공한 Service를 호출한다.

4.4 OCR 구현

UploadFile
→ 20MB / format / signature 검증
→ 임시 파일 저장
→ OpenCV 전처리
→ PaddleOCR PP-OCRv5
→ ProductCodeResolver
→ A의 ProductService.find_by_product_code()
→ DB 미일치/저신뢰 시 2차 전처리 + OCR
→ 결과 반환
→ OCR 원본 삭제

실제 품번 형식은 아직 모르므로 resolver.py 에 규칙을 모아두고 ProductService/Provider와 분리한다.

4.5 Try-on 구현

처음에는 실제 AI가 아니라 MockTryOnProvider .

```
POST /try-ons
  → File 저장 (원본 1h TTL)
  → Job PENDING 저장
  → BackgroundTask 등록
  → 202 반환

BackgroundTask
  → Job PROCESSING
  → MockTryOnProvider
  → 결과 파일 저장 (3h TTL)
  → TryOn 생성
  → Job SUCCEEDED
```

실제 모델이 정해지면 providers/try_on/real_xxx.py 만 추가한다.

4.6 추천 구현

MVP는 Rule-based.

```
Product.category
ProductTag(style/color/season)
재고 여부
  ↓
score 계산
  ↓
정렬
  ↓
회원 limit / 게스트 최대 3개
```

AI 추천 확장 시 RecommendationProvider 구현체만 변경한다.

5. 공유 영역 충돌 방지 규칙

공유 대상	Owner	규칙
app/main.py	A	B가 router 추가 필요 시 작은 PR
app/core/*	A	공통 변경은 먼저 합의
Product models/repository	A	B 직접 변경 금지, ProductService 사용
RegisteredProduct	A	B 진단은 registrationId만 사용
Store	A	B RepairService에서 StoreService 사용
FileMetadata	B	A 상품 이미지 저장이 필요하면 FileService 사용
Job	B	A에서 비동기 필요 시 JobService 사용
Alembic versions	순차	merge 시 한 head만 유지

6. 권장 Git Branch

```
main
|—— feature/be-a-auth-guest
|—— feature/be-a-product-cart
|—— feature/be-a-order-payment
|—— feature/be-a-owned-products
|
|—— feature/be-b-file-job
|—— feature/be-b-ocr
|—— feature/be-b-tryon
|—— feature/be-b-recommendation-coordi
|—— feature/be-b-diagnosis-repair
```

한 브랜치에 모든 기능을 몰아넣지 않는다. 작은 PR 단위로 main에 빨리 합친다.

7. 구현 순서 - 전체

Phase 0. 프로젝트 기반

A 중심, B 리뷰

1. FastAPI 프로젝트 생성
2. config / DB / SQLAlchemy Base
3. 공통 response / exception
4. JWT dependency 뼈대
5. Alembic 설정
6. /health 확인
7. Swagger 정상 동작

완료 기준: PostgreSQL 연결 + GET /health + migration 실행 성공.

Phase 1. 핵심 데이터 기반을 먼저 병렬 구현

A

1. User / RefreshToken
2. GuestSession / QR mapping / Store / Campaign
3. Product / ProductVariant / ProductTag / ProductImage
4. seed script
5. ProductService 계약

B

1. FileMetadata / LocalStorageService
2. 파일 validator
3. Job 모델/Service
4. BackgroundTask 공통 패턴
5. PaddleOCR Provider skeleton

통합 지점: B의 OCR이 A의 ProductService를 호출할 수 있어야 함.

Phase 2. 첫 번째 Vertical Slice

목표는 데모의 첫 핵심 경험을 최대한 빨리 끝까지 통과시키는 것이다.

게스트 생성
→ 품번 사진 업로드
→ OCR
→ Product 조회
→ 추천 3개

A

- POST /guest-sessions
- 상품 조회 API
- 최근 본 상품 저장

B

- POST /product-recognitions
- PaddleOCR + resolver
- RecommendationService

이 단계가 끝나면 프론트가 실제 사진을 찍어 상품을 찾을 수 있어야 한다.

Phase 3. 가상 피팅 / 코디

A

- Auth 완성
- Google/Kakao
- /auth/claim
- 회원 profile

B

- Avatar CRUD
- MockTryOnProvider
- /avatar-try-ons
- /try-ons
- Job polling
- Try-on 저장
- SavedCoordi CRUD

Vertical Slice

상품 인식
→ 내 사진 업로드
→ jobId
→ polling
→ Mock 합성 결과
→ 추천
→ 로그인
→ 결과/코디 저장

Phase 4. 장바구니 / 모의결제

A 주도

1. Cart / CartItem
2. Order / OrderItem
3. Payment

4. MockPaymentProvider
5. 재고/가격 검증
6. 주문 성공 시 RegisteredProduct 생성

B

- 필요한 Product/TryOn 결과 연결 검증
- 프론트 연동 지원

Vertical Slice

상품 → 장바구니 → 구매 → Mock 결제 성공 → 결제내역

Phase 5. 사후관리

A

- 기존 제품 시리얼 등록
- 내 제품 목록/상세
- ProductCareGuide

B

- Diagnosis Job
- MockDiagnosisProvider
- Damage 결과
- RepairReservation
- 수리예약 취소

Vertical Slice

구매 제품 또는 시리얼 등록 제품
 → 케어 정보
 → 사진/영상 진단
 → 결과
 → 수리 필요 시 예약

Phase 6. 최종 통합 / 검수

둘이 같이 한다.

1. API 명세와 Swagger 비교
2. 모든 endpoint response envelope 확인
3. camelCase 확인
4. KST +09:00 확인
5. 권한/소유권 테스트
6. 파일 TTL/cleanup 테스트
7. 게스트 Try-on 3회 제한
8. 실제 파일이 static 공개되지 않는지 확인
9. Mock payment 실패 시나리오
10. 서버 재시작 후 PROCESSING Job 정리
11. seed 데이터 기준 데모 전체 시나리오 실행

8. 개인별 구현 체크리스트

Backend A

- ☐ 프로젝트/core 기반
- ☐ PostgreSQL/Alembic
- ☐ User/RefreshToken
- ☐ GuestSession/QR mapping
- ☐ Google/Kakao OAuth
- ☐ Product/Variant/Tag/Image
- ☐ 최근 본 상품 DB
- ☐ Cart
- ☐ Order/OrderItem
- ☐ PaymentProvider/Mock
- ☐ RegisteredProduct
- ☐ ProductCareGuide
- ☐ Store/Campaign
- ☐ seed script
- ☐ A 담당 API integration test

Backend B

- ☐ FileMetadata/LocalStorage
- ☐ 파일 format/size/signature validator
- ☐ Job/BackgroundTasks
- ☐ PaddleOCR PP-OCrv5
- ☐ ProductCodeResolver
- ☐ Avatar
- ☐ MockTryOnProvider
- ☐ Try-on TTL/save
- ☐ RuleBasedRecommendation
- ☐ SavedCoordi
- ☐ MockDiagnosisProvider
- ☐ Damage/Diagnosis
- ☐

- RepairReservation
- CleanupService
- B 담당 API integration test

9. 최소 테스트 분담

A 테스트

회원가입/로그인/refresh/logout
 Google/Kakao 계정 분리
 게스트 QR/비QR 생성
 상품/variant 조회
 장바구니 가격/재고 검증
 Mock 결제 성공/실패
 주문내역
 시리얼 중복 등록

B 테스트

20MB 이미지 제한
 100MB 영상 제한
 잘못된 file signature
 OCR 미탐지/DB 미일치
 게스트 사진 Try-on 3회 제한
 Job 성공/실패/재시작 처리
 1h/3h TTL
 private file 소유권
 추천 게스트 최대 3개
 진단/수리예약 소유권

10. 가장 먼저 완성해야 할 데모 경로

기능을 전부 만든 뒤 통합하려고 하지 말고 아래 경로를 우선 완성한다.

1. 게스트 진입
2. 품번 사진 OCR
3. 상품 조회
4. 사진 Try-on Mock
5. 추천
6. 로그인
7. 장바구니
8. Mock 결제
9. 결제내역
10. 구매 제품 사후관리

이 경로가 통과하면 기획서의 핵심 가치인 **오프라인 제품 발견** → **디지털 체험** → **구매** → **사후관리**가 하나의 백엔드 흐름으로 연결된다.

11. 작업량 균형

Backend A	Backend B
인증 난이도 높음	OCR/파일 난이도 높음

Backend A	Backend B
상품/Variant 데이터 많음	BackgroundTask/Job 상태 관리
Cart/Order transaction	Try-on Provider/TTL
OAuth 2종	추천/코디
Mock Payment 확장 구조	Diagnosis/Repair
사후관리 보유제품 데이터	AI Provider 확장 구조

API 개수만 보면 B가 많아 보일 수 있으나 A는 인증/주문 transaction과 데이터 모델 제약이 복잡하다. 위 분할이면 실제 난이도와 병렬성이 비교적 균형적이다.

12. 병목이 생길 때 우선순위

시간이 부족하면 삭제하지 말고 Mock/Seed로 대체한다.

1. 실제 Try-on 모델 → Mock 유지
2. 실제 진단 모델 → Mock 유지
3. Google/Kakao 중 연동 이슈가 큰 쪽 → endpoint 계약 유지 + 테스트 provider
4. 수리 슬롯 복잡한 스케줄링 → seed slot 반환
5. 실제 품번 데이터 부족 → DEMO 품번 seed

반대로 아래는 생략하지 않는다.

- 인증/권한
- 제품 품번 OCR 흐름
- File private 접근
- Job polling
- 장바구니/주문 데이터 흐름
- Mock 결제
- 구매 후 RegisteredProduct 연결